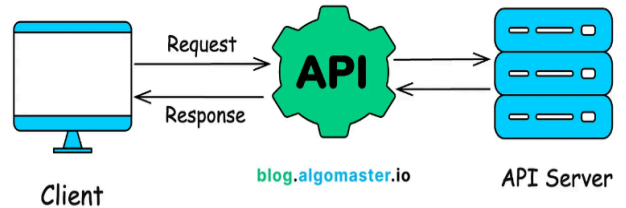


1. What is an API? (Core Concept)

Summary:

- API = Application Programming Interface.
- Acts as a **middleman** between two applications, enabling them to interact without sharing internal code or databases.
- Follows a **request-response model**: client → API → backend → response.



Examples:

- Weather API: input city → returns temperature, humidity.
- Uber API: input pickup/destination → returns driver & fare.
- Python's `sorted()` API: input list → returns sorted list.

Interview Tip:

“An API allows software systems to communicate with each other **via defined inputs and outputs**. It **abstracts complexity and ensures consistency across different clients**.”

For example, the Uber app uses APIs to find nearby drivers, calculate fares, and process payments.”

2. How APIs Power Modern Applications

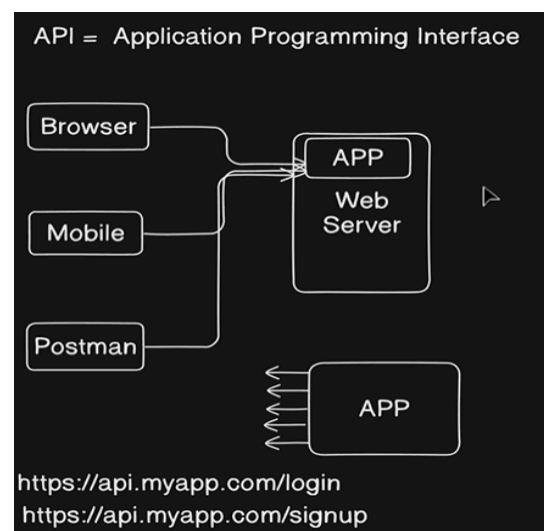
Api is a medium iske jriye hm apne application ki functionality ko expose krte hai

Summary:

An application is hosted on a server so it can be accessed by users. It typically consists of:

- **Backend APIs** – Handle business logic, data processing, and database communication.
- **Frontend UI** – Interacts with backend APIs EndPoints to provide a user-friendly interface.

Example – Uber:



- **Backend APIs:**
 - Find nearby drivers
 - Calculate fare and route
 - Process payments
 - Enable real-time tracking
- **Frontend:**
 - Sends requests to backend APIs
 - Displays driver info, fare, and ride status to users

Interview Tip:

“In real-world applications, backend APIs perform the heavy lifting—handling complex operations, calculations, and database interactions—while the frontend acts as an interface. For example, when you book a ride in Uber, the frontend calls multiple backend APIs to deliver a smooth and interactive experience.”

3. Types of APIs

Type	Audience	Use Case	Example
Open / Public API	External developers	Integration & innovation	YouTube Data API
Internal / Private API	Within organization	Efficient internal communication	Amazon order processing APIs
Code / Library API	Developers	Predefined functions in programming languages	Python <code>list.sort()</code> , TensorFlow API

Interview Tip:

“APIs are not just about connecting apps. There are public APIs for third-party integration, private APIs for internal systems, and code-level APIs that provide ready-made functions to simplify development.”

4. API Communication Methods

1. REST API

- **Definition:** Most widely used API style; lightweight, stateless, and scalable.
- **Communication:** Uses **HTTP methods** – GET (read), POST (create), PUT/PATCH (update), DELETE (remove).==>CRUD
- **Structure:** Resource-based; each resource has a **URL (endpoint)**.

- **Model:** Follows **client-server** model (request → response).
- **Data Format:** Usually **JSON**.

Example: Retrieve books from a bookstore

GET https://api.bookstore.com/books

Response:

```
[
  {"id":1,"title":"Clean Code","author":"Robert C. Martin"},
  {"id":2,"title":"The Pragmatic Programmer","author":"Andrew Hunt"}
]
```

2. SOAP (Simple Object Access Protocol)

- **Definition:** Older, XML-based API; more **structured and secure** than REST.
- **Use Cases:** Banking, healthcare, enterprise apps.
- **Structure:** Requires **WSDL file** to define available functions and request/response structure.
- **Data Format:** **XML**.

Example: Fetch account balance

```
<soapenv:Envelope ...>
  <soapenv:Body>
    <bank:GetAccountBalance>
      <bank:accountNumber>123456</bank:accountNumber>
    </bank:GetAccountBalance>
  </soapenv:Body>
</soapenv:Envelope>
```

3. GraphQL

- **Definition:** Flexible API; clients request **exact data needed**.
- **Communication:** Single endpoint; client sends **queries** specifying required fields.
- **Advantage:** Reduces multiple API calls compared to REST; more efficient for complex data.

Example: Fetch user profile & posts

```
{
  user(id: 123) {
    name
    email
    posts {
      title
      likes
    }
  }
}
```

```
}  
}  
}
```

- **Response:** JSON with only requested fields.

4. gRPC (Google Remote Procedure Call)

- **Definition:** High-performance API using **Protocol Buffers (Protobuf)** instead of JSON/XML.
- **Features:**
 - Binary format → smaller, faster payloads
 - Supports **bidirectional streaming** (client & server send data simultaneously)
 - Ideal for **microservices and real-time applications**
- **Example Use:** Real-time chat or video streaming services.

Quick Comparison Table:

Feature	REST	SOAP	GraphQL	gRPC
Data Format	JSON	XML	JSON	Protobuf (binary)
Endpoint	Multiple	Multiple	Single	Multiple
State	Stateless	Can be stateful	Stateless	Stateless
Use Case	Web/Mobile	Enterprise/Banking	Complex data apps	High-performance, microservices
Efficiency	Moderate	Low	High (fetch exact data)	Very High (binary + streaming)

5. Using an API – Step by Step

1. **Find an API** – public directories (RapidAPI, Postman, GitHub) or official docs.
2. **Read Documentation** – endpoints, inputs, outputs, authentication.
3. **Get Access** – API key, OAuth, JWT.
4. **Test API** – Postman or cURL.
5. **Write Code** – integrate in app using libraries (**requests** in Python, **fetch** in JS).
6. **Handle Errors** – invalid input, auth failures, rate limits.
7. **Use Response** – display data in frontend.

6. Key Takeaways for Interviews

- **API = Abstraction + Standardization** → lets apps communicate without exposing internal code.
- **Backend APIs handle logic**, frontend just interacts visually.
- **Types matter** → know public vs private vs library APIs.
- **Communication methods matter** → REST, SOAP, GraphQL, gRPC.
- **Practical knowledge** → being able to call APIs in code (Python/JS) and handle responses.

SpringBoot:

"Spring Boot automatically converts **Java objects to JSON** using **Jackson** through its auto-configuration, so developers don't need to explicitly handle serialization.

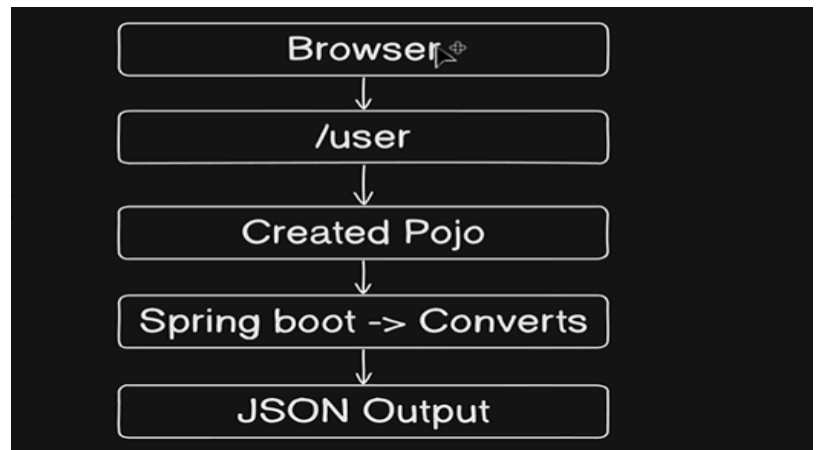
In contrast, Node.js frameworks like Express require explicit methods like `res.json()` to send JSON responses, giving more control but requiring more manual handling."

eg->

```
@GetMapping("/user")
public User getUser() {
    return new User("Satyam", 22);
}
```

Spring Boot automatically:

- Object → JSON convert karta hai
- Using **Jackson library** (included via starter)



"A POJO is a simple Java object used to model data, with private fields and public getters/setters. It's framework-independent and easy to use for passing data around."

Class jiski private fields ko hum use kr ske for modelling..

1. HTTP Methods (CRUD Mapping)

Method	Purpose
GET	Read data

POST	Create data
PUT	Update entire resource
PATCH	Update part of resource
DELETE	Remove resource

Use Mapping Annotations In SpringBoot

`@GetMapping, @PostMapping('/gng')...similarly`

Mapping:

1. @RequestMapping

- **Definition:** General-purpose annotation to map HTTP requests to controller methods in Spring MVC.
- **Usage:** Can specify:
 - **URL path** (`value` or `path`)
 - **HTTP method** (`method = RequestMethod.GET/POST/PUT/DELETE`)
 - **Headers, params** (optional)

Example:

`@RequestMapping(value = "/user", method = RequestMethod.GET)`

```
public String getUser() {
    return "User details";
}
```

- Here, `/user` endpoint responds to **GET requests**.

2. Shortcut Annotations

Instead of specifying `method` each time, Spring provides shortcuts:

Annotation	HTTP Method
<code>@GetMapping</code>	GET

@PostMapping	POST
@PutMapping	PUT
@PatchMapping	PATCH
@DeleteMapping	DELETE

Example:

```
@GetMapping("/user")
public String getUser() {
    return "User details";
}
```



3. Best Practices

Separate methods for each HTTP method – keeps code clean and clear.

```
@PostMapping("/user") // Create user
@GetMapping("/user") // Get user
```

Use @RequestMapping at class level for versioning

```
@RestController
@RequestMapping("/api/v1") // versioning
public class UserController { ... }
```

1.
 - Then method mappings are relative: `/api/v1/user`, `/api/v1/orders`, etc.

 **Interview Tip:**

“@RequestMapping maps HTTP requests to controller methods. For simplicity, we often use @GetMapping, @PostMapping, etc., and use class-level @RequestMapping for versioning APIs like `/api/v1`.”

What Happens Internally When Frontend hit endPoint?

Flow When a Request Hits a Spring Boot Endpoint

1. Browser / Postman

- The client sends an HTTP request (GET, POST, etc.) to your server.

2. Tomcat (Web Server)

- Spring Boot uses an embedded Tomcat server.
- Tomcat receives the HTTP request and passes it to **DispatcherServlet**.

3. DispatcherServlet

- Acts as the **front controller** in Spring MVC.
- Handles all incoming requests and delegates to appropriate handlers.

4. HandlerMapping

- DispatcherServlet asks the **HandlerMapping** which controller method matches the request URL and HTTP method.
- Finds the method annotated with **@GetMapping, @PostMapping, etc.**

5. Controller

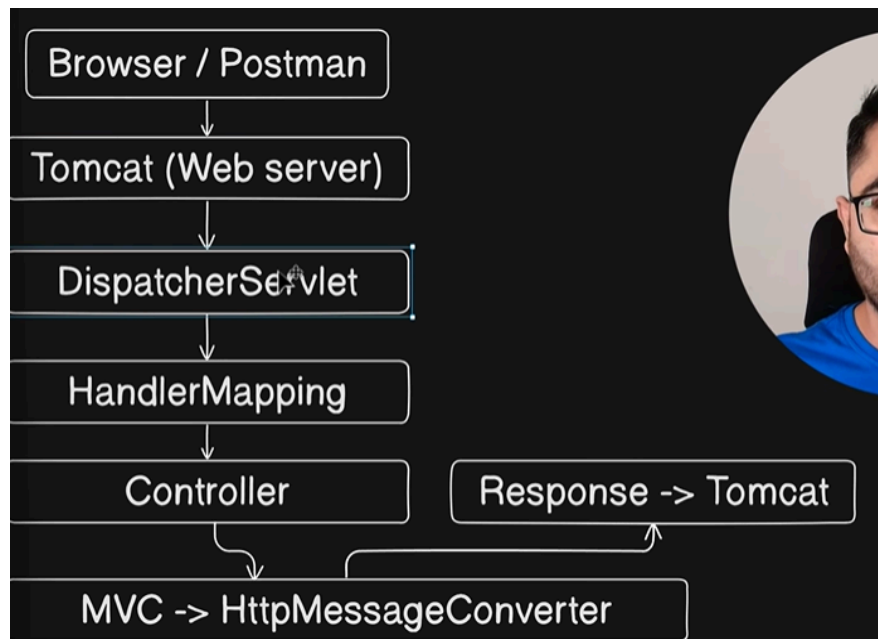
- The request is passed to the correct **controller method**.
- The controller executes **business logic**, interacts with services and repositories.

6. MVC → HttpResponseMessage

- After the controller returns data (like a POJO), Spring uses **HttpMessageConverter** to convert it into **HTTP response format**, e.g., JSON or XML.

7. Response → Tomcat

- The response is sent back through Tomcat to the **client** (browser or Postman).



Quick Summary:

Request → Tomcat → DispatcherServlet → HandlerMapping → Controller → HttpResponseMessage → Response → Client

- DispatcherServlet is the **central hub** --> delegates request to Appropriate Handler
- HandlerMapping decides **which controller handles the request**.
- HttpResponseMessage **serializes objects to JSON/XML** for HTTP responses.

 **Interview Tip:**

"In Spring Boot, when an endpoint is hit, the request goes to DispatcherServlet, which finds the matching controller method using HandlerMapping. The controller executes business logic, and the response is converted to JSON/XML via HttpMessageConverter before being sent back through Tomcat to the client."

What does Request Contains??

When we talk about an **HTTP request** in Spring Boot (or any web application), the request contains **all the information the server needs to process it**. Here's a breakdown of **what is inside a request**:

1. Request Line

- Contains the **HTTP method**, **URL**, and **HTTP version**.
- Example:

GET /api/v1/users/123 HTTP/1.1

- **GET** → HTTP method
- **/api/v1/users/123** → URL / endpoint
- **HTTP/1.1** → version

2. Headers

- Key-value metadata about the request.
- Common headers:
 - **Content-Type** → type of data sent (e.g., application/json)
 - **Accept** → expected response format (JSON, XML)
 - **Authorization** → access token or credentials
 - **User-Agent** → info about client/browser
 - **Cookie** → session info

Example:

Content-Type: application/json

Authorization: Bearer abc123token

3. Query Parameters (Optional)

Uses @RequestParam

- Extra info in the URL after ?
- Example:

GET /users?role=admin&active=true

- Here, `role=admin` and `active=true` are query params.

4. Path Variables

uses @PathVariable

- Dynamic parts of the URL, often used with REST endpoints.
- Example:

GET /users/123

- `123` can be captured as a **path variable** in Spring Boot with `@PathVariable`.

5. Request Body (Optional)

Uses @RequestBody

- Data sent by the client in **POST/PUT/PATCH requests**.
- Usually JSON or XML, mapped to a **POJO** in Spring Boot using `@RequestBody`.

Example:

```
{
  "name": "Alice",
  "email": "alice@example.com"
}
```

6. Cookies / Session Info (Optional)

- Cookies store session ID or user-specific data.
- Server can retrieve session data to identify the user.

Summary Table

Part	Purpose	Example
Request Line	Method, URL, version	<code>GET /users/123 HTTP/1.1</code>
Headers	Metadata	<code>Authorization: Bearer xyz</code>
Query Params	Optional filters/info	<code>/users?role=admin</code>
Path Variables	Dynamic URL values	<code>/users/123</code> → 123
Request Body	Data for POST/PUT/PATCH	<code>{ "name": "Alice", "email": "..."} </code>
Cookies / Session	Track user session / state	<code>JSESSIONID=abcd1234</code>

Interview Tip:

“An HTTP request carries everything the server needs: the method, URL, headers, optional query/path parameters, and body. Spring Boot maps headers, path variables, query params, and JSON bodies to method parameters or POJOs for processing.”

What is a Status Code?

A **status code** is a **numeric value sent by the server** in response to a client's HTTP request that tells the client what happened:

- **Success:** 2xx (e.g., 200 OK, 201 Created)
- **Client error:** 4xx (e.g., 400 Bad Request, 404 Not Found)
- **Server error:** 5xx (e.g., 500 Internal Server Error)

 **Interview Tip:** Always mention that status codes are **standardized in HTTP**, and in Spring Boot, they are represented as the **HttpStatus enum**.

2 Why Use ResponseEntity?

- By default, Spring returns **200 OK** for all responses if you just return a value (`public String getUser()`), even if it should be **201 Created** or **404 Not Found**.
- **ResponseEntity<T>** is a **wrapper** that gives you **full control over**:
 1. Status code

2. Response headers

3. Response body

3 How to Use It

Builder Style

```
@PostMapping("/createUser")
public ResponseEntity<User> createUser(@RequestBody User user) {
    UserDb.putIfAbsent(user.getId(), user);
    return ResponseEntity.status(HttpStatus.CREATED).body(user);
}
```

Another Way to use

```
return new ResponseEntity<>(user, HttpStatus.CREATED);
```

HttpStatus.(X)-->enum→jisme Status Value defined hai phle se hi

- `status(HttpStatus.CREATED)` → sets status 201
- `body(user)` → sets the response payload

4 Common HttpStatus Enums

Enum Name	Code	Meaning
<code>HttpStatus.OK</code>	200	Request successful
<code>HttpStatus.CREATED</code>	201	Resource created
<code>HttpStatus.NO_CONTENT</code>	204	Successful but no content
<code>HttpStatus.NOT_FOUND</code>	404	Resource not found
<code>HttpStatus.BAD_REQUEST</code>	400	Client error

Other helpful methods:

- `.build()` → builds a response without body
- `.notFound().build()` → returns 404
- `.noContent().build()` → returns 204

5 Interview-Friendly Explanation

“A status code is a numeric response from the server that indicates the result of a client’s HTTP request. In Spring Boot, we use the `HttpStatus` enum to represent these codes, and `ResponseEntity<T>` allows us to return the status code, headers, and body together in a single response. For example, returning `ResponseEntity.status(HttpStatus.CREATED).body(user)` sends a 201 status code along with the created user object.”

1 Dynamic URLs in Spring Boot

Dynamic URLs let you pass data in the URL itself or as query parameters.

There are three main ways:

1. **Path Variables** (`@PathVariable`) – part of the URL path.
2. **Query Parameters** (`@RequestParam`) – key=value pairs in the URL after `?`.
3. **Request Headers** (`@RequestHeader`) – sent as HTTP headers.

2 Path Variables (`@PathVariable`)

- Used to identify a specific resource.
- Mandatory by default (can be made optional with `required=false`).
- Can have multiple variables.

Example – Single PathVariable

```
@GetMapping("/user/{userId}")
public ResponseEntity<User> getUserById(@PathVariable("userId") int id){
    if(!UserDb.containsKey(id))
        return new ResponseEntity<>(HttpStatus.NOT_FOUND);
    return new ResponseEntity<>(UserDb.get(id), HttpStatus.OK);
}
```

- URL: `/user/3` → `id = 3`

Example – Multiple PathVariables

```
@GetMapping("/{userId}/orders/{orderId}")
public ResponseEntity<User> getUserOrder(@PathVariable int userId, @PathVariable int
orderId){
```

```

System.out.println("OrderId found: " + orderId);
if(!UserDb.containsKey(userId))
    return new ResponseEntity<>(HttpStatus.NOT_FOUND);
return new ResponseEntity<>(UserDb.get(userId), HttpStatus.OK);
}

```

- URL: `/3/orders/101` → `userId=3`, `orderId=101`

Optional PathVariable

```

@GetMapping("/user/{userId}")
public ResponseEntity<User> getUserById(@PathVariable(value="userId", required=false)
Integer id){
    if(id == null || !UserDb.containsKey(id))
        return new ResponseEntity<>(HttpStatus.NOT_FOUND);
    return new ResponseEntity<>(UserDb.get(id), HttpStatus.OK);
}

```

- If not provided, `id` can default to `null`.

3 Request Headers (@RequestHeader)

- Used to read **header information** from the HTTP request.
- Common headers: `User-Agent`, `Authorization`, `Content-Type`.

Example:

```

@GetMapping("/info")
public String getInfo(@RequestHeader("User-Agent") String userAgent){
    return "User Agent: " + userAgent;
}

```

- Retrieves the browser or client info sent in the request header.

4 Query Parameters (@RequestParam)

- Key=value pairs in URL after `?`.
- Used for **search, filtering, optional data**.
- Can be **mandatory or optional**.

Mandatory Query Param

```
// URL: /search?name=Satyam
@GetMapping("/search")
public ResponseEntity<List<User>> searchUsers(@RequestParam String name){
    List<User> users = UserDb.values().stream()
        .filter(u -> u.getName().equalsIgnoreCase(name))
        .toList();
    return ResponseEntity.ok(users);
}
```

Optional Query Param with Default

```
@GetMapping("/search")
public ResponseEntity<List<User>> searchUsers(@RequestParam(required=false,
defaultValue="Satyam") String name){
    List<User> users = UserDb.values().stream()
        .filter(u -> u.getName().equalsIgnoreCase(name))
        .toList();
    return ResponseEntity.ok(users);
}
```

Multiple Query Params

```
// URL: /search?name=Satyam&email=satyam@gmail.com
@GetMapping("/search")
public ResponseEntity<List<User>> searchUsers(
    @RequestParam(required=false, defaultValue="Satyam") String name,
    @RequestParam(required=false, defaultValue="default@gmail.com") String email
){
    List<User> users = UserDb.values().stream()
        .filter(u -> u.getName().equalsIgnoreCase(name))
        .filter(u -> u.getEmail().equalsIgnoreCase(email))
        .toList();
    return ResponseEntity.ok(users);
}
```

5 Combining PathVariable, RequestParam, RequestHeader

You can use all three in a single method:

```

@GetMapping("/user/{userId}/orders")
public ResponseEntity<User> getUserOrder(
    @PathVariable int userId,
    @RequestParam(required=false) String status,
    @RequestHeader("User-Agent") String userAgent
){
    System.out.println("User-Agent: " + userAgent);
    // filter by status if provided
    return ResponseEntity.ok(UserDb.get(userId));
}

```

6 Key Differences: PathVariable vs RequestParam

Feature	PathVariable	RequestParam
Location	Part of URL path	Query string after ?
Use case	Identify a specific resource	Filter or search resources
Optional?	Yes, with <code>required=false</code>	Yes, with <code>required=false</code>
Example URL	<code>/user/3</code>	<code>/search?name=Satyam</code>
Typical mapping	<code>@PathVariable</code>	<code>@RequestParam</code>

 **Tip:**

- Use **PathVariable** when resource is **mandatory** and uniquely identified by URL.
- Use **RequestParam** when data is **optional** or used for **filtering/searching**.

Extra Question:

1 Protocol Basics

Feature	HTTP	HTTPS
Port	80	443
Security	None (plain text)	Encrypted using TLS/SSL
TCP/UDP	Uses TCP only	Uses TCP only , with TLS handshake
Encryption	No	Yes (AES, RSA, etc.)
Data Integrity	None	Yes, via TLS

Key point: Both HTTP and HTTPS use **TCP**, not UDP. Browsers and servers establish a **TCP connection** before sending HTTP or HTTPS requests.

2 How TCP behaves (both HTTP and HTTPS)

- **TCP handshake:** `SYN → SYN-ACK → ACK`
- Once TCP is established:
 - HTTP sends plain text packets.
 - **HTTPS wraps the data in TLS, so the payload is encrypted.**

Difference in practice:

- Using **Wireshark**:
 - HTTP traffic shows URLs, headers, and body in **plain text**.
 - HTTPS traffic shows **encrypted blobs**; you cannot read the URL or content without decrypting TLS.

3 TLS/SSL in HTTPS (the extra step)

When you access `https://`:

1. **TCP handshake:** client ↔ server (just like HTTP)
2. **TLS handshake:**
 - Client requests secure connection
 - **Server sends certificate (proving identity)**
 - **Key exchange happens (RSA/ECDHE)**
 - **Both sides agree on a session key**
3. **Encrypted communication** using session key
 - **Actual HTTP request and response are encrypted over TCP**

So, **from TCP's perspective**, both HTTP and HTTPS are TCP streams. But HTTPS wraps data in encryption.

4 Practical differences in network experience

Scenario	HTTP	HTTPS
Packet capture	Can see all data, headers, cookies	Only encrypted bytes; headers/content hidden
Man-in-the-middle (MITM)	Easy to sniff/modify	Hard to modify; attacker sees encrypted data
Performance	Slightly faster (no TLS handshake)	Slightly slower due to TLS handshake/encryption
Browser behavior	Mixed content warnings if loaded insecure content	Secure padlock, trust verified via certificate

TCP vs UDP note:

- HTTP/HTTPS always use **TCP**.
- UDP is used in protocols like **DNS, video streaming, VoIP**, not typical web browsing.

Example Answer (Interview-ready)

“HTTP and HTTPS both use TCP for communication.

HTTP sends data in plain text over TCP port 80, so anyone capturing packets can read the request and response. **HTTPS adds a TLS/SSL layer over TCP** on port **443**, **encrypting the HTTP payload**. Practically, with HTTPS, **the TCP handshake is followed by a TLS handshake, after which data is encrypted, ensuring** confidentiality and integrity. In tools like Wireshark, HTTP content is readable, whereas HTTPS appears as encrypted blobs.”

Http->tcp

Https->tcp +tls/ssl certification adds